

1. Spiega la differenza tra i tre stream standard `stdin`, `stdout` e `stderr`.
Fornisci anche i rispettivi file descriptor.

`stdin` $\Rightarrow$ Standard Input, di default la tastiera (0)
`stdout` $\Rightarrow$ Standard Output, di default il terminale (1)
`stderr` $\Rightarrow$ Standard Error, di default il terminale (2)

2. Qual è la differenza tra gli operatori di redirectione `>` e `>>`? Fornisci un esempio pratico.

`>` $\Rightarrow$ Redirige lo `stdout` in modalità truncate (sovrascrive il file)
`>>` $\Rightarrow$ Redirige lo `stdout` in modalità append (aggiunge in coda al file)
Es:
`ls -l ~ > myfile.txt` (Stampa la Home directory nel file sovrascrivendo il contenuto)
`ls -l ~ >> myfile.txt` (Stampa la Home directory a fine del file senza sovrascrivere)

3. Illustra la differenza tra `comando1 ; comando2`, `comando1 && comando2` e `comando1 || comando2`.

`comando1 ; comando2` $\Rightarrow$ Esegue il comando 1 e il comando 2 in modo indipendente
`comando1 && comando2` $\Rightarrow$ Esegue il comando 1 e poi il comando 2 solo se il primo comando ha avuto successo (exit code 0)
`comando1 || comando2` $\Rightarrow$ Esegue il comando 1 e se non ha avuto successo allora esegue il comando 2 (exit code $\neq 0$)

4. Cosa fa il comando `echo ~` e in cosa differisce da `echo "Hello, world!" > myfile.txt`?

`echo ~` $\Rightarrow$ Stampa la Home Directory a schermo
`echo "Hello World" > myfile.txt` $\Rightarrow$ Stampa "Hello World" in `myfile.txt` in modalità truncate

5. Come funziona `cat -n file.txt` e in quale scenario può risultare utile?

`cat -n file.txt` $\Rightarrow$ mostra il contenuto del file numerando le righe (-n)
utile nei file di configurazione o nei log

6. Spiega le differenze tra more e less. Perché oggi si preferisce usare less?

more => Pagine che consente solo di andare avanti.

less => Pagine che consente di andare avanti, indietro, cercare testo... => Più utile di more

7. Descrivi le principali opzioni di sort e fornisci un esempio in cui l'opzione -k è indispensabile.

sort -n => ordinamento numerico sort -f => ordinamento case insensitive

sort -r => ordinamento inverso sort -k => ordina per un campo specifico

sort -t "separatore" => specificare il separatore (di base tab o space)

8. Qual è la differenza tra head -n 5 file.txt e tail -n +5 file.txt?

head -n 5 file.txt => Mostra le prime 5 righe di file.txt

tail -n +5 file.txt => Mostra dalla 5 riga in poi di file.txt

9. Perché il comando uniq richiede che le righe duplicate siano adiacenti? Come si può garantire ciò?

uniq => Elimina le righe duplicate adiacenti => Devono essere adiacenti per poter essere eliminate

sort file.txt | uniq => Elimina tutti i duplicati

10. Cosa restituisce il comando wc -lwm file.txt?

wc -lwm file.txt => Conta le righe (-l) parole (-w) e caratteri (-m) di file.txt

11. Con cut -f 2 -d ":" /etc/passwd, cosa ottieni?

cut -f 2 -d ":" /etc/passwd => Il secondo campo di ogni riga di /etc/passwd
risultato : come separatore

12. Qual è la differenza tra diff file1 file2 e diff -r dir1 dir2?

diff file1 file2 => Trova le differenze tra file1 file2 riga per riga

diff -r dir1 dir2 => Trova le differenze tra dir1 dir2 ricorsivamente anche nelle sotto directory

13. Scrivi il comando che, dato un file `elenco.txt`, visualizza la **terza parola della seconda riga**.

head -2 elenco.txt | tail -1 | cut -f 3 -d ""

↑ ↑ ↑

PRENDI IL PRIMO 2 PRENDI L'ULTIMA SCELGI IL 3 CAMPO (PAROLA)

RIGA DEL FILE RIGA DEL FILE

14. Spiega il significato dei seguenti metacaratteri nelle regex: $\wedge$, $\$$, $\cdot^*$, $a\{2,4\}$.

- $\wedge \Rightarrow$ Inizio Riga
- $\$ \Rightarrow$ Fine Riga
- $\cdot^* \Rightarrow$ Cerca qualsiasi espressione
- $a\{2,4\} \Rightarrow$ Cerca espressioni che contengano minimo 2 massimo 4 "a"

15. Quale differenza c'è tra `grep -E 'a+' file.txt` e `grep 'a\+' file.txt`?

grup -E 'a+' file.txt $\Rightarrow$ flag per le regex estese (non specific \ tra le regex)

grup 'a\+' file.txt $\Rightarrow$ grup classico

testuali: comando fanno la stessa cosa con un server locale con almeno una a

16. Spiega cosa fa il comando:

Bash Copia codice

```
grep '^1.*e$' file.txt
```

gruppo $\langle a, b \rangle$ che $\tau \in \tau$ è l'unico che inizia con a e finisce con b

17. Con `find . -type f -size +10M`, cosa stai cercando?

find -type f -size +20M => elenco : file normali (type f) nella directory corrente
(.) più grandi di 20 MB

18. Illustra con un esempio pratico l'uso di `sed -e 's/regex/sostituzione/g' file.txt`.

2da - e 's/regex / sostituzione / y' file.txt $\Rightarrow$ sostituzione tutte le occorrenze di regex con sostituzione in ogni riga del file

Es: `sed -e 's/foo/bar/g' file.txt` => sostituisce foo con bar in ogni riga!

1. Qual è la differenza tra `>` e `2>` in una redirectione? Fai un esempio pratico.

`>` => Reindirizzamento dello `stdout` in modalità troncata (2)
`2>` => Reindirizzamento dello `stderr` (2) ↑
DESCRIZIONE

2. Spiega cosa significa il comando:

Bash

 Copia codice

```
cmd > myfile.txt 2>&1
```

mette `stdout` di `cmd` in `myfile.txt` e lo concatena con lo `stderr`

3. Cosa fa la pipeline:

Bash

 Copia codice

```
ls -la | less
```

Mostra : file della directory corrente (-a include quelli nascosti con info aggiuntiva -l)
nel pannello less

4. Qual è la differenza tra `sort -n` e `sort -r`?

`sort -n` => Mette in ordine numerico

`sort -r` => Mette in ordine alfanumerico inverso

5. Come funziona `uniq -c` e in quale scenario può essere utile?

`uniq -c` => Mostra ogni riga con il numero di occorrenze accanto

Es: `sort myfile.txt | uniq -c` => mostra il quantitativo dei duplicati

6. Se un file ha 100 righe, cosa restituisce `head -n -10 file.txt`?

`head -n -10 file.txt` => Esclude le ultime 10 righe del file (100 -> 90 righe)

7. Con il comando `cut -c 1-5 file.txt`, cosa ottieni?

`cut -c 1-5 file.txt` ⇒ Estrae i caratteri dalla colonna 1 alla 5 di ogni riga

8. Spiega la differenza tra `diff file1 file2` e `diff --brief file1 file2`.

`diff file1 file2` ⇒ Differenza riga per riga tra file 1 e file 2

`diff --brief file1 file2` ⇒ Indica solo se i file sono diversi, senza dettagli

9. Cosa restituisce il comando:

Bash

 Copia codice

```
grep -E '^([0-9]){2}$' numeri.txt
```

`grep -E '^([0-9]){2}$' numeri.txt`

`-E` ⇒ Normalizzato esteso (senza `/`)

`^[0-9]` ⇒ Deve iniziare con un numero compreso tra 0 e 9

`{2}$` ⇒ Deve avere 2 cifre e fine riga

10. Che differenza c'è tra `grep 'pippo' file.txt` e `grep -v 'pippo' file.txt`?

`grep 'pippo' file.txt` ⇒ Cerca le corrispondenze esatte di 'pippo' in file.txt

`grep -v 'pippo' file.txt` ⇒ Mostra le righe che non contengono pippo

11. Con `find /home -type d -maxdepth 1`, cosa cerchi?

`find /home -type d -maxdepth 1` ⇒ Cerca le directory (`-type d`) nella cartella `/home` con profondità massima di 1 (è ferma alla 1° directory)

12. Cosa fa questo comando `sed -e '1,3d' file.txt`?

`sed -e '1,3d' file.txt` ⇒ Elimina dall'output le righe dalla 1 alla 3

13. Scrivi un comando che elimini tutte le righe che iniziano con `#` da un file di configurazione `conf.txt`.

`sed -e '/^#d' conf.txt`

14. Quale regex matcha una parola che inizia con "a" e termina con "z", indipendentemente da cosa c'è in mezzo?

guz -E ' ^ a . * z \$ ' ^

15. Con il comando `ls | sort | tail -1`, quale file viene mostrato?

- 1. Ultima fila ordinato delle directory corrente (Ordine lessicografico)

1. Spiega la differenza tra `stdout` e `stderr`. Perché a volte conviene separarli in due file diversi?

std::out = 0 gestione standard dell' output (terminale) (File Descriptor 1)

std error = 0 Lytane standard def: error (Terminals) (File Descriptor 2)

Comincio separare $std::out$ e $std::err$ per avere un maggior controllo sulla ridirigione dell'I/O (Nth in fase di debug)

2. Cosa significa `cmd < input.txt > output.txt 2> errori.txt`?

cmd < input.txt > output.txt 2> error.txt $\Rightarrow$ prende l'input da input.txt e
reindirizzandolo in output in output.txt (stampate), infine reindirizza lo stderr
in error.txt. qualora ci sia un errore.

3. Scrivi una pipeline che mostri solo la **prima parola dell'ultima riga** di un file `elenco.txt`.

tail -1 elements.txt | cut -d " " -f 1

↑
ULTIMA RIGA

↑
1° PAROLA CON SEPARATORE " "

4. Che differenza c'è tra echo ~ e echo "~"?

echo ~ => Stampo le Home directory dell'utente loggato in stdout

echo "~" => Stange la stringe "~" in std::ant

5. Cosa fa il comando `cat file1.txt file2.txt > unito.txt`?

cat file 1.txt file 2.txt > nullo.txt => Mette il contenuto di file 1.txt e file 2.txt in nullo.txt in modalità troncato (sovrascrivibile)

6. Perché less è preferibile a more per leggere file lunghi?

less => Navigatore che m avanti e indietro, volendo puoi avere occorrenze (+ V) etc)
more => Navigatore che m solo avanti

7. Quale comando mostra le prime 15 righe di un file?

head -n 15 nomefile.txt
↑
more 15 more

8. Scrivi il comando che stampa le righe dalla 10 alla 20 di un file testo.txt.

head -n 20 testo.txt | tail -n 11
↑ ↑
more 20 more le ultime 11

9. Se un file contiene righe duplicate non consecutive, perché conviene usare sort | uniq invece di uniq da solo?

sort | uniq => Combinazione che elimina tutti i duplicate
uniq => Elimina solo i duplicate adiacenti

10. Spiega cosa restituisce wc -l file.txt.

wc -l file.txt => Mostra il numero di righe del file file.txt

11. Con cut -d ":" -f 1 /etc/passwd, cosa ottieni?

cut -d ":" -f 1 /etc/passwd => Ottengo la 1 versione di ogni riga (Separatore :)

12. Qual è la differenza tra diff file1 file2 e colordiff file1 file2?

diff file1 file2 => Mostra le differenze riga per riga di file1 e file2
colordiff file1 file2 => Mostra le differenze riga per riga di file1 e file2
visualizzabile con colori

13. Scrivi una regex che seleziona tutte le stringhe che iniziano con una lettera maiuscola e terminano con un punto.

"^[A-Z].*\$"
↑
\o per indicare il punto

14. Cosa fa il comando `grep -v '^#' conf.txt`?

`grep -v '^#' conf.txt` => Mostra le righe che non iniziano con #

15. Scrivi un comando `find` che cerchi tutti i file `.txt` più grandi di 1 MB nella directory corrente e sottodirectory.

`find . -type f -name "*.txt" -size +1M`

16. Con `sed -e 's/[0-9]//g' testo.txt`, cosa ottieni?

`sed -e 's/[0-9]//g' testo.txt` => Elimina tutte le cifre numeriche del file

17. Qual è la differenza tra `ls | sort | head -1` e `ls | sort | tail -1`?

`ls | sort | head -1` => 2: mostra il primo elemento della lista di file ordinata

`ls | sort | tail -1` => 2: mostra l'ultimo elemento della lista di file ordinata

18. Scrivi un comando che mostri solo le righe di `dati.txt` che contengono esattamente 3 cifre consecutive.

`grep -E '[0-9]{3}' dati.txt`

1. Qual è la differenza tra > e >>? Porta un esempio.

> => Ridireziona lo stdout in modalità truncata (sovrascrive il contenuto)
>> => Ridireziona lo stdout in modalità append (aggiunge il contenuto e fine file)
n.b.) File Descriptor stdout = 1

2. Cosa fa il comando:

Bash

 Copia codice

```
cmd > out.txt 2>&1
```

cmd > out.txt 2>&1 => Ridireziona lo stdout di cmd in modalità truncata concatenando anche lo stderr (file descriptor: 2)

3. Con comando1 && comando2, in quali casi viene eseguito comando2?

comando1 && comando2 => Esegue il comando 2 solo se il primo ha avuto successo!

4. Scrivi un comando che mostri solo la terza riga di file.txt.

```
head -n 3 file.txt | tail -n 1
```

5. Cosa fa cat -n file.txt rispetto a cat file.txt?

cat => Converte e mostra su stdout l'output di 1+ file
cat -n => Similitudine di cat + Numerazione delle righe in output

6. Perché less è più comodo di more?

Il navigatore less è più comodo di more poiché esso permette di scorrere avanti e indietro e consente anche di ricercare del testo con / testo.

7. Con sort -k 2 -t ":" elenco.txt, cosa succede?

sort -k 2 -t ":" => Ordina in base al 2 campo specificando il separatore ":" con -t

8. Cosa fa il comando tail -n +5 file.txt?

tail -n +5 => Esclude le prime 5 righe di file.txt

9. Perché `uniq` richiede righe duplicate adiacenti?

`uniq` elimina / mostra solo i duplicati adiacenti, se vogliamo vedere / eliminare tutti i duplicati dobbiamo usare `sort file.txt | uniq`

10. Cosa restituisce `wc -lw file.txt`?

`wc -lw file.txt` ⇒ Mostra le righe (`-l`) e le parole (`-w`) del file `file.txt`

11. Che differenza c'è tra `cut -c 1-3 file.txt` e `cut -f 1 -d ":" file.txt`?

`cut -c 1-3 file.txt` ⇒ Estrae i primi 3 caratteri di ogni riga

`cut -f 1 -d ":" file.txt` ⇒ Estrae il primo campo di ogni riga (`-d ":"` separatore)

12. Spiega l'utilità di `diff -r dir1 dir2`.

`diff -r dir1 dir2` ⇒ Trova le differenze in maniera ricorsiva di `dir1` e `dir2`

13. Scrivi un comando che mostri solo le righe di `file.txt` che terminano con `".txt"`.

`grep -E '\.txt$' file.txt`

14. Qual è la differenza tra `grep -i "ciao" file.txt` e `grep "ciao" file.txt`?

`grep -i "ciao"` ⇒ Trova le corrispondenze case insensitive di `ciao`.

`grep "ciao"` ⇒ Trova le corrispondenze esatte di `ciao`.

15. Con `find . -type f -name "*.sh"`, cosa cerchi?

`find . -type f -name "*.sh"` ⇒ Cerco i file che finiscono con `.sh` nella directory corrente

16. Cosa fa il comando `sed -e 's/foo/bar/g' testo.txt`?

`sed -e 's/foo/bar/g' testo.txt` ⇒ Sostituisce tutte le occorrenze di `foo` con `bar` nel file

17. Scrivi un comando che elimini la prima riga di elenco.txt.

```
tail -n +2 elenco.txt > temp.txt && mv temp.txt elenco.txt
```

18. Quale regex matcha una stringa composta solo da cifre?

"^[0-9]+\$" oppure "[0-9]+\$"

13. Spiega la differenza tra > e >> nella redirectione.

> => Redirige lo std out in modalità troncatura (File descriptor 2)

>> => Redirige lo std out in modalità appenda (File descriptor 2)

14. Cosa fa il comando `cmd > out.txt 2>&1`?

`cmd > out.txt 2>&1` => Redirige lo std out in `out.txt` e aggiunge in quel file anche lo standard error

15. Scrivi una pipeline che mostri le ultime 5 righe del file `log.txt` ordinate alfabeticamente senza ripetizioni.

```
sort log.txt | uniq | tail -n 5
```

16. Descrivi il funzionamento dei comandi `head`, `tail` e `cut`.

`head` => Mostra le prime n righe di un file (Default 10 righe)

`tail` => Mostra le ultime n righe di un file (Default 10 righe)

`cut` => Mostra determinati campi di ogni riga di un file

17. Scrivi un'espressione regolare che individui stringhe contenenti almeno tre cifre consecutive.

'[0-9]{3,}'

18. Qual è la differenza tra `grep -E`, `grep -i` e `grep -v`?

`grep -E` => Usa regex estese

`grep -i` => Mostra le occorrenze che matchano la regex (case insensitive)

`grep -v` => Mostra le occorrenze che non matchano la regex

13. Quali sono i tre file descriptor standard (numeri e significato)?

0 $\Rightarrow$ stdin : Input standard della shell (tastiera)
1 $\Rightarrow$ stdout : Output standard della shell (terminale)
2 $\Rightarrow$ stderr : Error standard della shell (terminale)

14. Spiega la differenza tra ; , && e || nell'esecuzione di comandi multipli.

;
=> esegui i due comandi in modo indipendente
&&
=> esegui il comando 2 solo se il comando 1 ha successo (exit code 0)
||
=> esegui il comando 2 solo se il comando 1 e' fallito (exit code != 0)

15. Scrivi una pipeline che conti il numero di righe duplicate in un file dati.txt.

```
sort dati.txt | uniq -d | wc -l
```

16. A cosa servono i comandi uniq -d e uniq -u?

uniq -d
=> Mostra solo le righe duplicate
uniq -u
=> Mostra solo le righe uniche

17. Scrivi un comando che visualizzi solo le prime 20 righe di un file testo.txt numerate.

```
cat -n testo.txt | head -n 20
```

18. Spiega la differenza tra le regex a*, a+, a? e a{2,4} con un esempio.

a*
=> 0 o più occorrenze di a
a+
=> almeno una o più a
a?
=> al massimo una a
a{2,4}
=> Da 2 a 4 occorrenze di a consecutive

13. Cosa fa il comando `cmd < input.txt > output.txt 2> error.txt`?

Prende l'input da `input.txt`, mette l'output in modalità troncata in `output.txt` e mette gli errori in `error.txt`

14. Scrivi una pipeline che mostri l'ultima riga del file `nomi.txt` in ordine alfabetico.

```
sort nomi.txt | tail -n 1
```

15. Qual è la differenza tra i comandi `less` e `cat`?

`less` => Paginatore che consente di andare avanti e indietro e anche di cercare nel testo
`cat` => Mostra il contenuto di uno o più file

16. Scrivi una regex che individui tutte le righe che iniziano con una lettera maiuscola.

```
'^[A-Z]'
```

17. A cosa serve l'opzione `-n` del comando `grep`?

`grep -n` => Mostra il numero delle righe di dove si trova l'occorrenza

18. Qual è la differenza tra `wc -l`, `wc -w` e `wc -c`?

`wc -l` => Conta le righe di un file
`wc -w` => Conta le parole di un file
`wc -c` => Conta i Byte di un file

13. Spiega la differenza tra `>` e `>>` in una redirectione.

`>` ⇒ Redirecta lo `stdout` in modalità truncata
`>>` ⇒ Redirecta lo `stdout`

14. Cosa fa la pipeline `dmesg | grep usb | less`?

`dmesg`: Mostra i messaggi del kernel
`grep usb`: Mostra le occorrenze che contengono `usb`
`less` ⇒ Seguire per l'output

15. Scrivi un comando che mostri solo le ultime 15 righe del file `error.log` numerandole.

```
tail -n 15 error.log | cat -n
```

16. Qual è la differenza tra `cut -c 1-5 file.txt` e `cut -f 1 -d"," file.txt`?

`cut -c 1-5 file.txt` ⇒ prende i primi 5 caratteri di ogni riga
`cut -f 1 -d","` ⇒ prende il primo campo con separatore `","`

13. Cosa fa il comando `sort -r file.txt | uniq`?

Mette in ordine alfabetico inverso il contenuto del file `file.txt` senza duplicati

14. Qual è la differenza tra `tee` e la semplice redirectione `>`?

`tee` scrive sul file mentre `>` redirecta e basta

15. Scrivi una pipeline che mostri i primi 10 processi ordinati per utilizzo di CPU (usando `ps` e `sort`).

```
ps aux | tr -s " " | sort -k 3 -t " " -r | head -n 10
```

16. Quale differenza c'è tra i metacaratteri ^ e \$ nelle regex?

^ => Deve iniziare con

\$ => Deve finire con

17. Scrivi una regex che individui indirizzi email con dominio .it.

"[a-zA-Z0-9._%+-]+@([a-zA-Z0-9-]+\\.)*[a-zA-Z0-9-]+\\.it\$"

18. Cosa fa l'opzione -c del comando grep?

grep -c => Mostra la riga che corrisponde al pattern